MIS41110 Practical Week 2, Solutions

Covers the Week 2 lecture (output, input, files, arguments, functions, modules) · **Worked** Monday of Week 3

Exercise 1: arguments math

```
# Week-2 sheet, exercise 1 -- pair math from program arguments.
# (c) sys.argv[0] is the program's own name; the real arguments start at [1].
# (d) The program can COMPUTE a decision this week and cannot ACT on it: that
#     needs if, which arrives on Friday. Same move as cafe_w2.py's big_order.
import sys

a = float(sys.argv[1])
b = float(sys.argv[2])

print(f"{a} + {b} = {a + b}")
print(f"{a} - {b} = {a - b}")
print(f"{a} * {b} = {a * b}")
print(f"{a} / {b} = {a / b}")
print(f"{a} ** {b} = {a ** b}")

big_pair = a > 10.0 and b > 10.0
print(f"Big pair (both over 10)? {big_pair}")
```

(a) Any two numbers work because nothing is hard-coded.

(b) f-strings replace week 1's `str()` gluing: the value goes inside the braces and Python formats it.

(c) `sys.argv[0]` is the program's own name, so the first real argument is `[1]` and the second is `[2]`.

(d) `big_pair = a > 10.0 and b > 10.0` prints `False` for `13 5`: the first value clears 10, the second does not, and `and` needs both. The register can compute the decision and cannot act on it, which is exactly where `cafe_w2.py` left off.

Exercise 2: sign the crew in

```
# Week-2 sheet, exercise 2 -- crew_add.py: append one crew line per run.
import sys

filename = sys.argv[1]
name = input("Name: ")
role = input("Role: ")

crew_file = open(filename, "a")          # "a": add to the end, never wipe
crew_file.write(name + " " + role + "\n")
crew_file.close()
```

Exercise 3: read the roster back

```
# Week-2 sheet, exercise 3 -- crew_list.py: print the roster back.
# (a) ex2 needed "a" (append across runs); here "r" (read only).
import sys

filename = sys.argv[1]
crew_file = open(filename, "r")
contents = crew_file.read()
crew_file.close()

print(f"Crew on file '{filename}':")
print(contents, end="")                  # lines already carry their own \n
```

(a) `crew_add.py` needed `"a"`, which adds to the end and keeps what is already there. `crew_list.py` needs `"r"`, which only reads. Opening with `"w"` would empty the file before you read a byte of it.

Exercise 4: the supply table

```
# Week-2 sheet, exercise 4 -- the aligned supply table.
item1 = "fuel drum"
weight1 = 180.0
item2 = "ration box"
weight2 = 22.5
item3 = "med kit"
weight3 = 9.75

print(f"{'ITEM':<12} {'WEIGHT KG':>9}")
print(f"{item1:<12} {weight1:>9.2f}")
print(f"{item2:<12} {weight2:>9.2f}")
print(f"{item3:<12} {weight3:>9.2f}")
```

Exercise 5: the station math module

`station_math.py`:

```python
# Week-2 sheet, exercise 5 -- the station math module.

def c_to_f(celsius):
    """Celsius to Fahrenheit."""
    return (9 / 5) * celsius + 32


def wind_chill(temp_c, wind_kmh):
    """The standard wind-chill formula (temperature in C, wind in km/h)."""
    return 13.12 + 0.6215 * temp_c - 11.37 * wind_kmh ** 0.16 + 0.3965 * temp_c *
        ↪ wind_kmh ** 0.16


def split_shift(hours):
    """Return full 4-hour watches and the leftover hours -- two values."""
    return hours // 4, hours % 4


def is_freezing(celsius):
    """True below zero -- the comparison already IS the boolean."""
    return celsius < 0
```

`tonight.py`:

```python
# Week-2 sheet, exercise 5 -- tonight.py: importing the station module.
# (a) tonight.py only calls the functions; better internals change nothing here.
# (b) No: variables born inside wind_chill are LOCAL and vanish when it returns.
import station_math

outside = -17.1
wind = 35.0
duty_hours = 11

watches, spare = station_math.split_shift(duty_hours)

print(f"Outside: {outside} C = {station_math.c_to_f(outside):.2f} F")
print(f"Freezing: {station_math.is_freezing(outside)}")
print(f"Feels like: {station_math.wind_chill(outside, wind):.1f} C in the wind")
print(f"Watches: {watches} full, {spare} hours spare")
```

(a) `tonight.py` only calls the functions by name. Improving what happens inside one of them changes nothing on the calling side, which is the whole point of putting it in a module.

(b) No. A variable created inside `wind_chill` is local: it exists while the function runs and vanishes when it returns. Only the returned value crosses back.

Meaningful names, a comment where a line needs one, run after every change.